Lab 1

The modern graphics pipeline runs entirely on the GPU. Because of this, we cannot just call a simple function to draw something; we first have to tell the GPU what our data is and what to do with it.

This usually happens in three main steps:
1. Allocating GPU memory and moving our data to the GPU
2. Telling our GPU what to do with this data
3. Calling draw commands

Our goal is to draw an orange triangle that fills the whole window. OpenGL only draws what exists between −1 and 1 in the xy plane (This is called the NDC coordinate system, more on that later...)

We will start by defining vertices of the triangle

```
float vertices[] = {
  0.5f, 1.0f, 0.0f,
  -1.0f, 0.0f, 0.0f,
  1.0f, 0.0f, 0.0f
};
```

This triangle will fill our whole screen. These vertices exist on our RAM, the first thing to do is to move them to the GPU for drawing.

Allocating GPU Memory

We will start by allocating GPU memory and move the data there and this is done first by telling OpenGL that we need some memory, and this "some memory" is called a *Buffer*, specifically a *Vertex Buffer* since it stores vertices.

This tells OpenGL that we will be needing some memory space later,

```
unsigned int buffer_id;
glGenBuffers(1, &buffer_id);
```

OpenGL functions are a bit weird, they don't take destinations as parameters, instead we have to **bind** our destination and OpenGL keeps track of what object is currently bound and uses it as a destination, so first we have to bind our newly created buffer before moving data to it.

```
glBindBuffer(GL_ARRAY_BUFFER, buffer_id);
// The second parameters is the amount of bytes we are moving, and the third parameter is
where we are reading from.
glBufferData(GL_ARRAY_BUFFER, 3 * 3 * sizeof(float), vertices, GL_STATIC_DRAW);
```

Now the first part is finished, our vertices are in our GPU memory.

Telling our GPU what to do with our data

Currently our GPU has our vertices, it does not know how to read the bytes we gave to our GPU. Currently it only sees 36 raw bytes, it does not know they are 3 vertices where each one is made up of 3 floats.

The first step here is to create a VAO, a VAO could be thought of as a container that stores metadata on how to interpret the raw bytes in the vertex buffer. It has at least 16 slots to store metadata for different VBOs

```
unsigned int VAO;
glGenVertexArrays(1, &VAO);
```

Same thing again, before using the VAO we bind it, but also the VAO has to know which VBO we are talking about so we will also bind the VBO.

```
glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, buffer_id);
```

Now we will "create" the metadata for our VBO.

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
```

The first function says the following: store the following metadata at slot 0, for the currently bound VBO, read them as 3 components (x, y, z), each of them is of a type float, don't normalize, and the next vertex will be at `3 * sizeof(float)`, start reading from byte 0.

The second function call is just enabling this slot to be used now.

Shaders

Shaders are in the most simple definition, programs that run on the GPU, they are heavily parallelized, this is the way we can draw 3D objects huge number of times per second. These shaders are written in a different language called *GLSL*, it is very similar to C. Shaders will be explained with more details in lab 2.

Calling draw commands

After setting up the shaders, the final thing to do is to draw. We are drawing very frame, so we will be having a `while` loop that is constantly drawing

The condition in the loop checks if the close button has been pressed, if it is not it continues drawing the current frame. *Swaps the buffers* and polls the events, the events are our mouse and keyboard inputs.

Finally we will be adding a function that draws, and since the very basic shape is a triangle, we will be calling `glDrawArrays(GL_TRIANGLES, 0, 3)`, which draws triangles for three vertices, starting from the first vertex.

```
while (!glfwWindowShouldClose(window)) {
  glDrawArrays(GL_TRIANGLES, 0, 3);

  glfwSwapBuffers(window);
  glfwPollEvents();
}
```